

Synchronization Support in 64-bit Out-Of-Order Superscalar Dual-Core RISC-V Processor

*

Shubham Yadav

Dept. of Electronic System Engineering
Indian Institute of Science, Bangalore
Bengaluru, India, 560012
shubhamyadav@alum.iisc.ac.in

Manish Kumar

Dept. of Electronic System Engineering
Indian Institute of Science, Bangalore
Bengaluru, India, 560012
manishkumar5@alum.iisc.ac.in

Sajin S

Dept. of Electronic System Engineering
Indian Institute of Science, Bangalore
Bengaluru, India, 560012
sajins@iisc.ac.in

Shubham Sunil Garag

Dept. of Electronic System Engineering
Indian Institute of Science, Bangalore
Bengaluru, India, 560012
shubhamsunil@iisc.ac.in

Kuruvilla Varghese, Senior Member, IEEE

Dept. of Electronic System Engineering
Indian Institute of Science, Bangalore
Bengaluru, India, 560012
kuru@iisc.ac.in

Abstract—This paper discusses the implementation of atomic instructions in a dual-core 64-bit out-of-order superscalar processors based on the open-source RISC-V instruction set architecture. Leveraging the advantage of RISC-V's modularization characteristics, each core implements RV64IMAFDC extension and optional supervisor and user mode privilege levels. In this paper, we focus on the A-extension, the atomic instruction set extension. This extension introduces instructions that provide atomic memory operations, enabling synchronization across multiple RISC-V harts within the same memory space. Our goal is to present an efficient execution flow of atomic memory operation instructions and Load-Reserved/Store-Conditional instructions for a dual-core System-on-Chip. We subsequently verify the synchronization capabilities through the execution of a standalone game application on SoC implemented on a Xilinx Kintex UltraScale KU085 FPGA-based board.

Index Terms—RISC V, Out of order, AMO, LR/SC, MESI, SCU, Synchronization, Superscalar, Multi-Core Processor.

I. INTRODUCTION

RISC-V is an open source instruction set architecture (ISA) developed by the University of California, Berkeley, based on the reduced instruction set computer (RISC) principle [1], [2]. The popularity of RISC-V is increasing due to its modular ISA, and compact instruction set encoding. Multi-core processors can execute multiple tasks or threads simultaneously, dividing the workload among the cores. Many modern applications, such as web browsers, media players, and scientific simulations, can use thread-level parallelism. Multi-core processors provide a natural platform for exploiting such parallelism, leading to faster and more responsive execution. In-order multicore architecture simplifies the processor design and control logic but may lead to low performance due to

pipeline stalls resulting from instruction dependencies. Out-of-order multi-core processors offer significant performance advantages by concurrently executing multiple instructions. Multicore processors require effective synchronization mechanisms.

RISC-V ISA A standard extension defines two types of synchronization instructions: atomic memory operations (AMO) and load reserved and store conditional (LR/SC) [1]. AMO and LR/SC instructions read, modify, and write memory atomically to synchronize several RISC-V harts in the same memory space. RISC-V supports the release consistency model and FENCE instruction to enforce memory ordering in the program flow. This paper proposes a solution for synchronization in dual-core RISC-V processors using the A-extension of ISA.

Section II discusses the multicore processing and Section III briefs about the processor architecture [3] and changes around it to support A-extension of ISA. Section IV describes the design of Load Store Unit (LSU) and data memory subsystem to support A-extension. Section V discusses the design changes of AXI-lite private bus of dual core. Section VI describes the design of snooping control unit (SCU) to support A-extension and Section VII discusses about the dual core SoC. Section VIII discusses the dual core synchronisation testing. Section IX briefs about the results and Section X concludes.

II. MULTICORE PROCESSING

A multicore processor is a multiprocessing system where two or more processing units called cores, each of which can read and execute instructions, are replicated and fabricated on a single integrated circuit. Such processors can run instructions concurrently on separate cores, increasing the overall performance of programs that support multi-threading

or parallel computing techniques. Processors used in all personal computers today are multi-core processors. Most of today's multi-core processors are symmetric (shared-memory) multiprocessors (SMPs) having the number of cores sharing a single centralized memory that all processors have equal access. This architecture is called uniform memory access (UMA) multiprocessors since all processors have a uniform latency for main memory access. Multicore SoC presented in this paper uses two symmetric cores.

A critical section is a piece of code that should be executed under mutual exclusion. If two threads access the same data in the global address space without synchronization, a deadlock or a race condition can occur. To avoid these conditions OSs use semaphore or mutex mechanisms.

To implement these synchronisation mechanisms different types of instructions are employed by various ISAs. RISC-V ISA provides two types of instructions which can implement these synchronisation mechanism i.e. Atomic memory operation (AMO) and Load-reserved and store conditional (LR/SC) instruction.

Linux compatible multicore designs must support cache coherency, consistency and synchronisation mechanism for proper operation. This paper presents the design of RISC-V multicore synchronisation.

III. PROCESSOR ARCHITECTURE AND SYNCHRONISATION

Figure 2 shows the microarchitecture of our dual core processor. Both cores are identical and hence can be used for symmetric multi-processing. Each core implements a nine stage pipeline: Fetch, Precode, Checking, Decode, Rename, Dispatch, Issue, Execute and Retire [3]. Both the cores are multi-layered which support user, supervisor and machine modes and thus are Linux compatible. Instructions go through all the pipeline stages and instructions are executed by different functional units. Figure 1 shows the execute stage of both the cores. The execution stage [7] splits instruction-specific execution unit into multiple functional units (FU), each performing specific operation as shown in Figure 1. These are integer ALU, integer multiplier, floating point (FP) ALU, FP multiplier. These functional units are packed into one execution unit. Both the AMO and LR/SC instructions are handled by the store functional unit. Atomic instructions supports acquire (*aq*) and release (*rl*) bits which helps to implement release consistency. Decode stage of both the cores are hardwired for *aq* and *rl* bits to be 1 which implements total ordering in the design. Hence when decode unit decodes an atomic instruction then it first waits for all subsequent instructions to retire and dispatches the atomic instruction. Once the atomic instruction is dispatched then the decode unit stops dispatching other instructions until atomic instruction retires. Multicore design requires support for synchronisation, cache coherency, and inter-processor interrupt. Dual core processor supports cache coherency and interprocessor interrupt. Figure 2 shows the design to support atomic instruction for synchronisation.

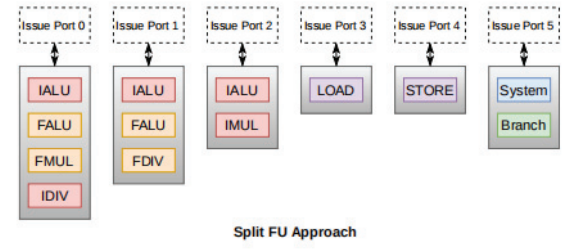


Fig 1. Execution Unit

Execution flow of AMO instruction in store functional unit generates AMO request to load-store unit which services this request by disabling the interrupts and gives this AMO request to D-cache. All AMO operations are done in D-cache. D-cache hardware loads the cache line, do the AMO operation and stores back the result in the D-cache itself. D-cache has been designed as discussed in Section IV to handle AMO operations [3].

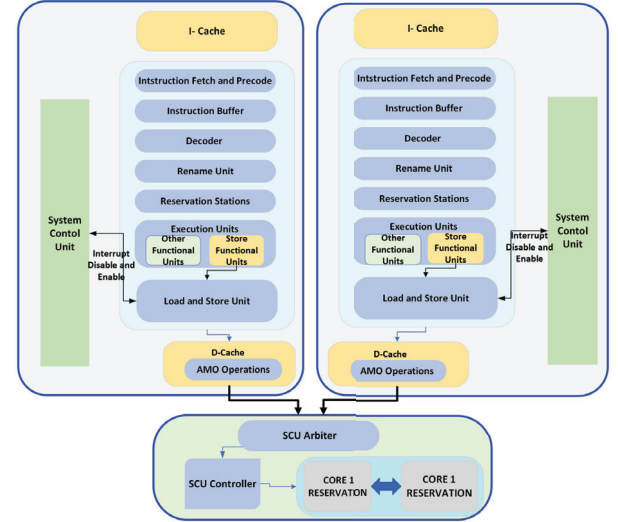


Fig 2. Microarchitecture of dual core processor

For LR/SC instruction, reservation registers are implemented in snooping control unit as shown in Figure 2. Execution flow of LR/SC instruction is same as AMO instruction till load-store unit. When D-cache receives an LR/SC request from the load store unit, it generates a request to snooping control unit (SCU) which has reservation set registers for both the cores. SCU puts reservation of the respective core and invalidates the reservation of the other core, if it has reserved the same address for LR instruction. The SC instruction checks for valid reservation done by the same core and if a valid reservation is found, it sends back D-cache line write signal and do not allow SCU to service a request from the other core until SC instruction completes. Figure 12 shows the block diagram of the dual core SoC. Each core has two buses which are AXI-Lite and AXI-full. AXI-Lite is used for core local interrupt controller (CLIC), platform level interrupt controller

(PLIC) and CoreCFG registers and AXI4 bus is used to communicate to rest of the SoC. I.e to memory subsystem and I/O peripherals. For AXI4 bus, cache-coherency and consistency is implemented using snooping control unit and AXI-Lite bus uses a simple arbiter to impart consistency in global memory ordering. Sections IV and Section V discusses on memory subsystem and AXI-Lite modifications in detail.

Execution flow of LR/SC instruction of each core is as follows. Store functional unit send LR/SC request to load store unit (LSU). The LSU stores SC request in its store queue and disable the interrupt. D-cache receives request from LSU and it sends reservation request to SCU. For LR instruction, SCU puts reservation on the requested memory address. For SC request, SCU invalidates the reservation of the corresponding core which generated the SC request and sends back pass/fail signal to D-cache if there is no valid reservation. D-cache returns normal load completion in case of LR instruction and for SC instruction it returns pass or fail signal. Store functional unit receives LR/SC request response from LSU and it send it to reorder buffer (ROB) which retires these instruction. As soon as LSU receives retire completion for SC from the retire unit, it removes the SC request from its store queue and enables the interrupts.

Execution flow of AMO intruction of each core is as follows. Store functional unit sends AMO request to LSU. The LSU treats AMO request as store request and store it into store queue and disables the interrupt. D-cache receives request from LSU and does the AMO operation on its cache line and stores back the result. Store functional unit receives the response of AMO instruction from LSU and send it to ROB. LSU receives retire request from retire unit and removes AMO request from its store queue and enables the interrupt.

IV. LOAD STORE UNIT AND COHERENT DATA MEMORY SUBSYSTEM

The datapath of load store unit (LSU) [4] is shown in Figure 3. When the store functional unit in the execution unit sends AMO request or SC instruction, LSU sends this request to its store buffer. AMO and SC requests are stored in the store buffer without both instructions being retired and interrupt is disabled. For all other instructions, store requests are serviced by LSU only when they are retired and interrupt is not disabled. When store buffer services SC or AMO instructions, it disables the interrupt and waits for a response from D-cache and when it receives response, it sends this response back to the execution unit. When AMO or SC instruction retires, LSU enables interrupt and removes that instruction from its buffer. LSU treats LR instruction as normal load instruction.

Multicore subsystem requires cache coherency. Snooping based cache coherency protocol has been implemented in this design. Design of data memory subsystem is shown in Figure 4. The D-cache is 16 KiB, 4-way set associative, physically indexed physically tagged (PIPT) with a block size of 32-bytes. Each way in the D-cache has 128 sets. Hence, physical address bits [11:5] index the cache, and bits [4:0] act as offset

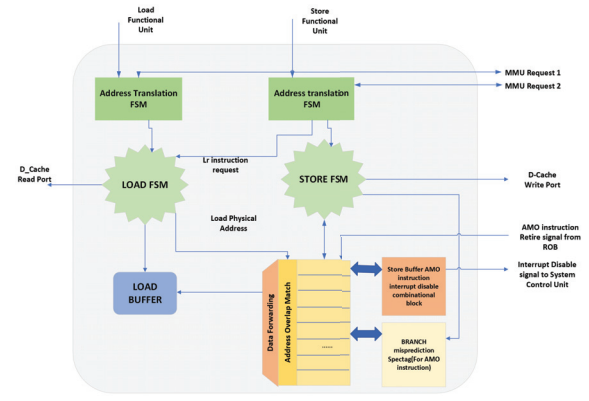


Fig 3. Load store unit

bits. As the maximum addressable physical address in RISC-V is 56-bit, bits [55:12], 44-bits form the tag. A cache line consists of a physical tag and 256-bits data block. Data cache follows write-back policy, i.e., data is written to the main memory only when modified cache line is replaced. Hence, when a cache line with MESI bits in modified state only will be written to the main memory during its replacement. Psuedo least recently used (PLRU) algorithm is used to determine the way to be replaced in case of replacement. D-cache subsystem uses dual port RAM where one port is used to serve read and write request from processor side, and the second port is dedicated to snooping request from the bus side. Two MESI bits represent the state of the cache line.

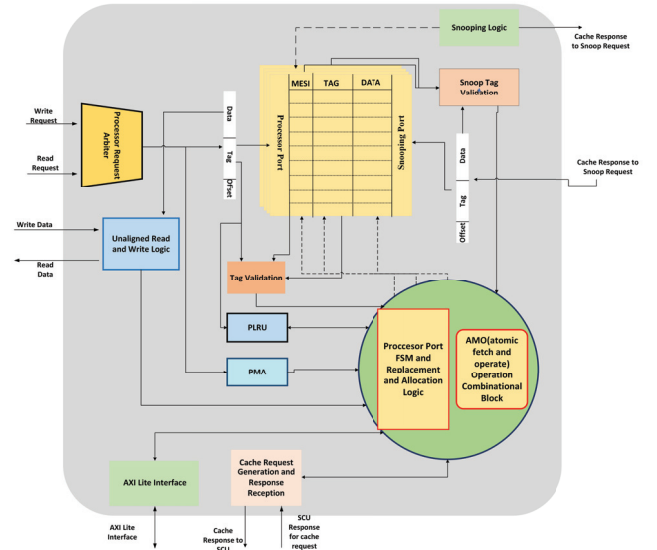


Fig 4. Architecture of dual core for Synchronization

As can be seen in Figure 4, AMO circuit [5] is implemented in the D-cache itself. When D-cache services the AMO request, it first loads the cache line, if it is a miss then it loads the cache line using SCU. AMO circuit does the operation. After storing the result back in the cache line, D-cache sends back the completion signal to load-store unit which send it back to

higher pipeline stages and enables the interrupts when AMO instruction retires. If the second core snoops the same memory location before the first core finishes writing the cache line, then the first core invalidates its entry and again tries to load the new value of the cache line and do its atomic operation and stores back the result. For LR/SC instruction, D-cache first generates a snooping request to SCU to put a reservation for LR instruction and to check and invalidate the reservation if it is an SC instruction. D-cache stores the SC request if SCU sends valid reservation signal to D-cache, otherwise D-cache sends failure signal back to higher pipeline. LR/SC instructions initiate a bus request to SCU as shown in state diagram of D-cache subsystem in Figure 5. In addition to this, for SC instruction, arbiter of SCU does not receives request from other core until the current SC request is completed by the D-cache of respective core which will be discussed in sections VI.

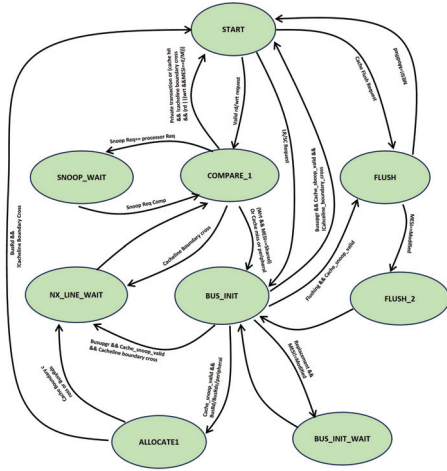


Fig 5. FSM Diagram of D-Cache Subsystem

V. PRIVATE BUS IN MULTICORE ARCHITECTURE

Our multicore processor design has platform-level interrupt controller (PLIC) and core local interrupt controller (CLINT) on the private bus as shown in Figure 6. Private bus of dual core uses AXI-Lite bus protocol. AXI-Lite protocol does not provide locking mechanism [6]. AXI-Lite interface is provided by D-cache for communicating with memory mapped hardware internal to the processor core.

The physical memory attributes (PMA) defines the property of each region of physical address space. PMA are hardcoded for the current design. Hence, physical memory regions of the memory mapped registers can be hardcoded to support atomicity on private bus provided physical memory protection registers has been configured for atomicity in this region.

Private bus is 32 bits wide, but some memory mapped registers like *mtime* and *mtimecmp* are 64 bits wide for 64-bit architecture. To support 64-bit write on these registers, two bus cycles are needed. Local memory ordering and global memory

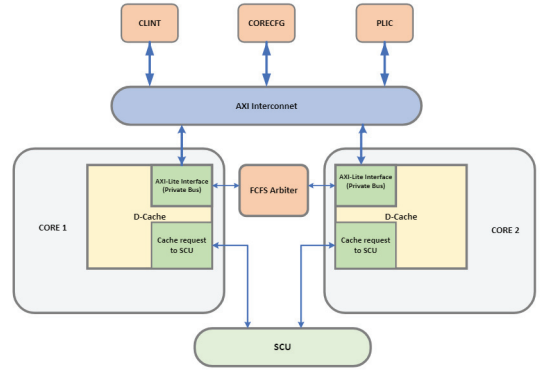


Fig 6. Private bus

ordering can be different for 64-bit write as shown in Figure 7.

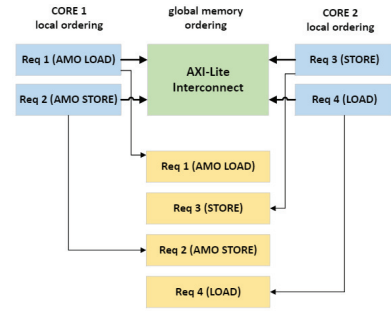


Fig 7. Local and global memory ordering before addition of arbiter

As can be seen from Figure 7, AMO request generated by core 1 may have undesirable ordering in global memory ordering seen by both the cores. This design uses a arbiter between both the cores to maintain correct global memory ordering. AXI-Lite interface of both the cores first send the request to arbiter which is a first come first serve arbiter and upon getting an *allow* signal, it sends the data on private bus. For AMO and 64-bit data requests, the arbiter does not give access of private bus to other core until the current core completes AMO instruction or 64-bits data write. Figure 8 shows the local and global memory ordering after adding arbiter in the design. AMO operation are done in D-cache itself. D-cache loads the data from private bus peripherals using AXI-Lite interface, performs the the AMO operation in D-cache and stores the result back to private bus peripherals. During this operation, arbiter does not give access to the private bus to other core.

Figure 9 shows the state diagram of AXI-Lite bus. When *start-burst* signal which represent a AXI-Lite transaction is valid then *start* state is changed to *arbite*. In *arbite* state, core waits for permission to use AXI-Lite interface for private bus from a first come first serve (FCFS) arbiter which is common to both the cores. Arbiter does not allow other core to access private bus until the current core finishes its AMO instruction of double word access to memory, since private bus is of 32

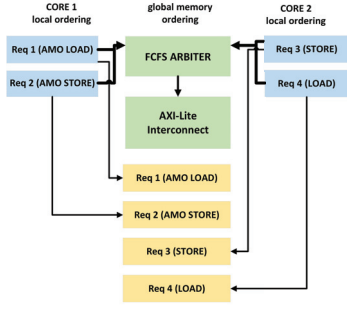


Fig 8. Local and Global memory ordering after addition of arbiter

bits only. For load and store instructions, it jumps to *private bus* state. For AMO instruction, it moves to *amo read* state. The *private bus* state services normal load and store memory requests. When *mem_ack* signal is received, this state moves to *private bus done*. The *private bus done* state checks if load or store request is a word request or double word request. If it is a double word request, then it moves to *private bus* state to fetch 32-bit MSB of memory region otherwise it moves to *start* state for completion. In *amo read* state, word size memory location is read and after reading it jumps to *amo* state. In *amo* state AMO operation are done on the loaded memory value and it moves to *private bus* state to store back the result to memory location.

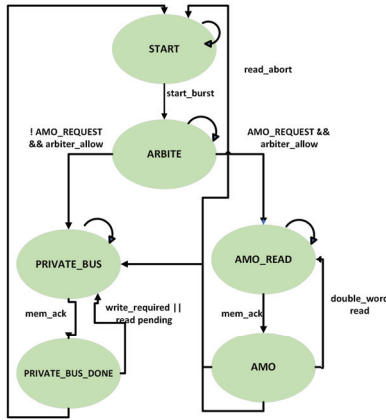


Fig 9. AXI-Lite bus controller

VI. SNOOPING CONTROL UNIT

Snoop Control Unit (SCU) is the hardware that coordinates snooping operations between multiple cores. The current SCU can handle four cores and can support more cores with minor changes in the design. Implementing a reservation set registers [4] in SCU will also ensure LR/SC reservation as per global memory ordering. When there is miss in D-cache or D-cache requests a cache line, then D-cache sends a request to SCU through its cache request port [3]. SCU has a first come first serve arbiter which services the request of cache of each core. When multiple caches generates request in the same clock

cycles, a request from processor core with lower hart id is given the priority. Every D-cache responds to SCU's request via its snoop request port in two clock cycles after checking the requested cache line status. Reservation set register has been added in SCU for LR/SC instruction for dual core as shown in Figure 10. D-cache sends request to SCU to put a reservation on a given address for LR instruction. If other core has already put a reservation on same address in its reservation set registers, then that reservation will be invalidated and reservation is made for most recent LR instruction in global memory order.

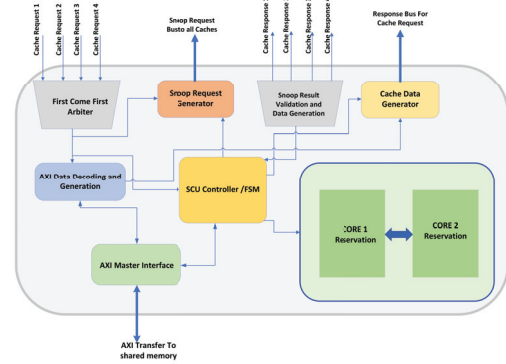


Fig 10. Snooping Control Unit (SCU) design

For SC instruction, D-cache sends request to SCU, SCU checks for valid reservation on that address. If there is a valid request, then SCU invalidates that reservation and sends valid reservation signal to D-cache. Arbiter of SCU do not service other core request till SC instruction stores its data in the cache. If SCU reservation is invalid, then SCU sends an invalid reservation signal through its cache response signals and D-cache does not store the SC request and sends *sc failed* signal to the higher pipeline.

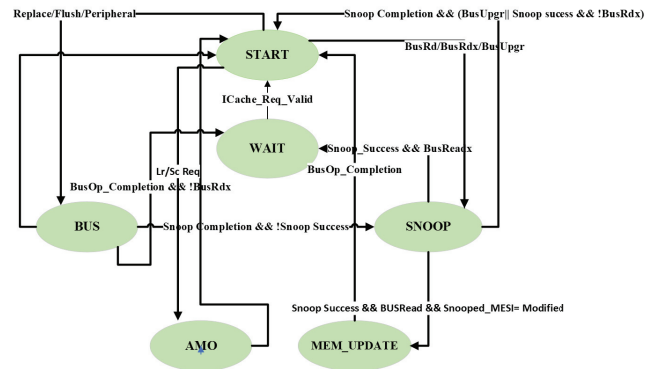


Fig 11. Snooping Control Unit (SCU) FSM

Figure 11 shows the state diagram of the controller of SCU. For LR/SC request the FSM moves to *amo* state from *start* state. Upon *BusR/BusRdx/BusUpgr* request FSM moves to *snoop* state. For *Replace/Flush/Peripheral* request it jumps to *bus* state. In *snoop* state SCU sends a snoop request to

all the caches and waits for their responses. For successful snoop response and request is *BUS Readx*, it jumps to *wait* state. If the snoop response is not successful, then it jumps to *bus* state. The *snoop* state is accessed if snooping fails or if the requested address is not cacheable. The *mem update* state is used to update main memory if snooping request is in a modified state. For LR/SC request, reservation is either put or invalidated which is done in *amo* state.

VII. DUAL CORE SoC

The dual core processor and the SoC [3] around it as shown in Figure 12. Dual core processor has a private bus which uses AXI-Lite bus specifications. Private bus has CLINT, PLIC and CoreCFG. CoreCFG are custom memory mapped registers. Both the cores are connected to SoC bus via SCU for data cache coherency. SoC bus has SRAM and BootROM connected to an AXI crossbar switch. ZSBL in BootROM supports loading binary image of application program to SRAM via UART XMODEM protocol and execution from SRAM. CoreCFG implements a memory-mapped core-to-core register for communication between cores. ZSBL can use these registers for dual core booting.

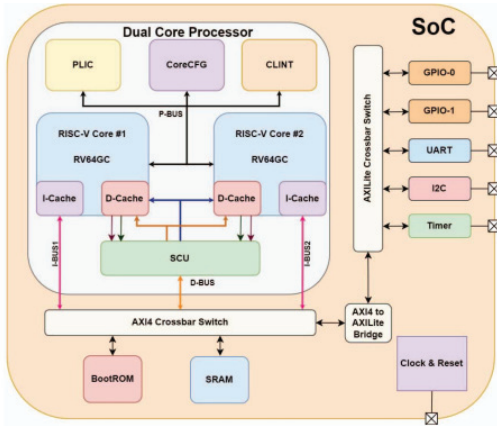


Fig 12. SoC architecture around dual core processor

SoC also has AXI4 to AXI-Lite bridge which connects AXI4 interconnect to AXI-Lite crossbar switch. AXI-Lite crossbar switch connects various peripherals like uartlite, quad spi, timer, gpio, etc. Memory map of dual core SoC is shown in Table 1.

VIII. VERIFICATION

Synchronization has been tested by running two parallel threads on each core such that each core fights for the lock and once a core gets the locks, it executes the code in the critical region and releases the lock. Figure 13 shows the structure of each thread. Thread has lock acquire, critical section and lock release code. The critical section code is common to both the thread and is on shared memory. In our tests, we have written console print codes in the critical section of code. Figure 14 and 15 shows the console output before and after the synchronization. Figure 14 shows that before synchronization

TABLE I
MEMORY MAP OF THE SYSTEM

Base	Top	Description	Bus
0x00001000	0x000013FF	CoreCFG (1 KiB)	Private
0x00010000	0x00013FFF	BootROM (16 KiB)	AXI
0x02000000	0x0200FFFF	CLINT (64 KiB)	Private
0x0C000000	0x0CFFFFFF	PLIC (16 MiB)	Private
0x10000000	0x1000FFFF	Internal SRAM (64 KiB)	AXI
0x20000000	0x20000FFF	UART (4KiB)	AXILite
0x20010000	0x20010FFF	GPIO0 (4KiB)	AXILite
0x20020000	0x20020FFF	I2C (4KiB)	AXILite
0x20030000	0x20030FFF	Timer (4KiB)	AXILite
0x20050000	0x20050FFF	GPIO1 (4KiB)	AXILite

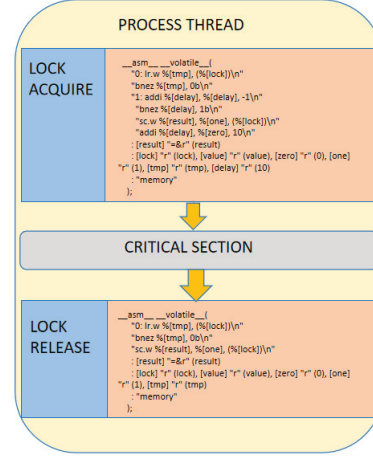


Fig 13. Structure of thread

both cores got the lock and printed logs simultaneously but after synchronization, both cores printed logs correctly.

```
oreC1o rge0t glooctk
lcoocrke
lc orreel0e arseilnegas iln0gc k1
ock
```

Fig 14. Console output before synchronization

```
Core1 got lock
core1 releasing lock
Core0 got lock
core0 releasing lock
```

Fig 15. Console output after synchronization

In addition to this in order to test the functionality of the dual-core processor synchronization, a **tic-tac-toe** game has been designed that is played by both cores such that before accessing the shared data, the respective core locks the critical section so that the other processor cannot modify shared data. Flow diagram as shown in Figure 14 explains the working of both the cores.

The synchronization functionality of the dual-core processor has been tested by executing AMOSWAP and LR/SC instructions. The game is played by both cores, hence, both fight for

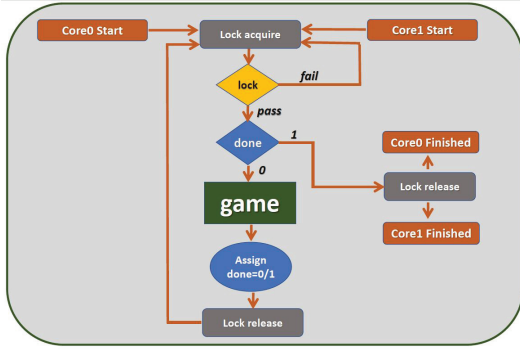


Fig 16. Flow Diagram for tic-tac-toe game

the lock and the winner locks the critical section and proceeds with the game. Other processor will try continuously. Once a core releases the lock, again both cores fight for lock and the game continues until it results in a win or a draw.

IX. TIMING, RESOURCE UTILIZATION AND BENCHMARKING RESULTS

Our SoC has been tested on XCKU085-FLVA1517 Kintex Ultrascale FPGA at a clock frequency of 16 MHz. Connection from issue unit to dispatch unit was the critical path limiting the clock speed. But at 16 MHz clock frequency, the single core and the dual core have setup slack of 7.95 ns and 0.177 ns. Dual core processor is benchmarked using Dhrystone, and a benchmark score of 1.724 DMIPS is obtained. It has also passed all the tests for RISC-V processors [7], [8]. The current dual-core processor is implemented on HTG-K800 development board which uses XCKU085-FLVAL1517 FPGA from Xilinx. Post-Implementation resource utilization for the current design is shown in Table 2.

TABLE II
RESOURCE UTILIZATION OF DUAL-CORE PROCESSOR

Resource	Utilization	Available	Utilization %
LUT	450442	497520	90.54
LUTRAM	548	267840	0.20
FF	108893	995040	10.94
BRAM	204	1620	12.59
DSP	40	4100	0.98
IO	24	624	3.85
BUFG	10	1128	0.89
MMCM	1	22	4.55

X. CONCLUSION

Synchronisation of our dual core 64-bit RISC-V, out of order, superscaler processor is discussed in this paper. The processor core implements RISC-V RV64IMAAFD extension and has been tested both by RISC-V processor tests [7] and bare metal custom c code. The proposed dual core SoC can be tested for dual core booting by modifying the DRAM and FLASH memory interface in the SoC using appropriate DRAM and flash controllers. The proposed SoC uses homogenous RISC-V core which can be used for symmetric multiprocessing but proposed synchronisation design can be

implemented for heterogenous core also where both cores are not identical. Vector extension can be added for enhanced computing.

REFERENCES

- [1] Andrew Waterman, and Krste Asanović, "The RISC-V Instruction Set Manual Volume I: Unprivileged ISA" Document Version 20191213, Technical report, RISC-V Foundation, December 2019.
- [2] Andrew Waterman, and Krste Asanović, "The RISC-V Instruction Set Manual Volume 11: Privileged Architecture," Document Version 20190608-Priv-MSU-Ratified, Technical report, RISC-V Foundation, June 2019.
- [3] Sajin S, Shubham Sunil Garag, Anuj Phegade, Deepshikha Gusain, and Kuruvilla Varghese "Design of a Multi-Core Compatible Linux Bootable 64-bit Out-of-Order RISC-V Processor Core", 2023 36th International Conference on VLSI Design and 2023 22nd International Conference on Embedded Systems (VLSID), 2023, Hyderabad, India, 8-12 Jan 2023.
- [4] Anuj Phegade and Deepshikha Gusain. Linux Compatible 64-bit Out of Order Superscaler RISC-V Processor: Masters Thesis. Department of Electronic Systems Engineering, Indian Institute of Science, June 2021.
- [5] Tuan Ta, Lin Cheng, and Christopher Batten, "Simulating Multi-Core RISC-V Systems in gem5" 2nd Workshop on Computer Architecture Research with RISC-V (CARRV-2), June 2018.
- [6] AMBA AXI and ACE Protocol Specification." Technical report, ARM, 2013.
- [7] "Tests for RISC-V Processors," <https://github.com/riscv/riscv-tests>.
- [8] OpenSBI. <https://github.com/riscvsoftware-src/opensbi>.